17. KZG 커밋먼트 구현 분석

개요

이 문서는 Lighthouse에서 구현된 KZG (Kate-Zaverucha-Goldberg) 커밋먼트 시스템을 상세히 분석합니다. KZG 커밋먼트는 Ethereum의 Deneb 업그레이드(EIP-4844)에서 도입된 핵심 암호학적 기법으로, 블롭(blob) 데이터의 무결성을 보장하고 효율적인 데이터 가용성 샘플링을 가능하게 합니다.

Lighthouse의 KZG 구현은 `crypto/kzg` 모듈에 위치하며, C-KZG 라이브러리와 Rust-ETH-KZG 라이브러리를 기반으로 구축되어 있습니다. 이 구현은 블롭 처리, 데이터 가용성 샘플링(DAS), 그리고 PeerDAS 프로토콜을 지원합니다.

KZG 모듈 구조

핵심 컴포넌트

```
crypto/kzg/
├── src/
| ├── lib.rs # 메인 KZG 래퍼 구조체
| ├── kzg_commitment.rs # KZG 커밋먼트 타입 정의
| ├── kzg_proof.rs # KZG 증명 타입 정의
| └── trusted_setup.rs # 신뢰할 수 있는 설정 관리
├── benches/
| └── benchmark.rs # 성능 벤치마크
├── Cargo.toml # 의존성 정의
```

```
└── trusted_setup.json # 신뢰할 수 있는 설정 파라미터
```

주요 의존성

```
[dependencies]
c-kzg = { workspace = true } # C-KZG 바인딩
rust_eth_kzg = { workspace = true } # Rust KZG 구현 (PeerDAS용)
ethereum_ssz = { workspace = true } # SSZ 직렬화
tree_hash = { workspace = true } # 머클 트리 해싱
serde = { workspace = true } # JSON 직렬화
```

KZG 커밋먼트 구현

KzgCommitment 구조체

```rust
// crypto/kzg/src/kzg_commitment.rs
#[derive(Derivative, Clone, Copy, Encode, Decode)]
#[derivative(PartialEq, Eq, Hash)]
#[ssz(struct_behaviour = "transparent")]
pub  struct KzgCommitment(pub [u8; c_kzg::BYTES_PER_COMMITMENT]);

impl KzgCommitment {
/// 버전화된 해시 계산 (EIP-4844)
```

```rust
pub fn calculate_versioned_hash(&self) -> Hash256 {
let mut versioned_hash = hash_fixed(&self.0);
versioned_hash[0] = VERSIONED_HASH_VERSION_KZG; // 0x01
Hash256::from_slice(versioned_hash.as_slice())
}
/// 테스트용 빈 커밋먼트 생성
pub fn empty_for_testing() -> Self {
KzgCommitment([0; c_kzg::BYTES_PER_COMMITMENT])
}
}
```

주요 특징:

- **크기**: 48바이트 (BLS12-381 G1 포인트)
- **버전화된 해시**: EIP-4844에서 정의된 형식으로 해시 계산
- **SSZ 직렬화**: Ethereum 표준 직렬화 지원
- **Tree Hash**: 머클 트리 구성을 위한 해싱 지원

KzgProof 구조체

```rust
// crypto/kzg/src/kzg_proof.rs
#[derive(PartialEq, Hash, Clone, Copy, Encode, Decode)]
#[ssz(struct_behaviour = "transparent")]
pub struct KzgProof(pub [u8; BYTES_PER_PROOF]);
```

```rust
impl KzgProof {

/// 무한대 점을 사용한 유효한 빈 증명 생성

pub fn empty() -> Self {

let mut bytes = [0; BYTES_PER_PROOF];

bytes[0] = 0xc0; // G1_POINT_AT_INFINITY 마커

Self(bytes)

}

}
```

주요 특징:

- **크기**: 48바이트 (BLS12-381 G1 포인트)
- **빈 증명**: 무한대 점으로 초기화된 유효한 증명
- **타입 안전성**: 강타입 래퍼로 타입 안전성 보장

신뢰할 수 있는 설정 (Trusted Setup)

TrustedSetup 구조체

```rust
// crypto/kzg/src/trusted_setup.rs

#[derive(Debug, Clone, PartialEq, Serialize, Deserialize)]

pub struct TrustedSetup {

#[serde(rename = "g1_monomial")]

g1_monomial_points: Vec<G1Point>,

#[serde(rename = "g1_lagrange")]
```

```rust
g1_points: Vec<G1Point>,
#[serde(rename = "g2_monomial")]
g2_points: Vec<G2Point>,
}

impl TrustedSetup {
pub fn g1_points(&self) -> Vec<[u8; BYTES_PER_G1_POINT]> {
self.g1_points.iter().map(|p| p.0).collect()
}
pub fn g2_points(&self) -> Vec<[u8; BYTES_PER_G2_POINT]> {
self.g2_points.iter().map(|p| p.0).collect()
}
}
```

구성 요소:

- **G1 Monomial Points**: 단항식 기저의 G1 포인트들
- **G1 Lagrange Points**: 라그랑주 기저의 G1 포인트들
- **G2 Monomial Points**: 단항식 기저의 G2 포인트들

신뢰할 수 있는 설정 로딩:

```rust
pub const TRUSTED_SETUP_BYTES: &[u8] = include_bytes!("../trusted_setup.json");

pub fn get_trusted_setup() -> Vec<u8> {
TRUSTED_SETUP_BYTES.into()
```

```
}
```

메인 KZG 래퍼 구조체

Kzg 구조체

```
// crypto/kzg/src/lib.rs
#[derive(Debug)]
pub struct Kzg {
trusted_setup: KzgSettings, // C-KZG 설정
context: DASContext, // PeerDAS 컨텍스트
}

impl Kzg {
/// 신뢰할 수 있는 설정으로부터 KZG 인스턴스 생성
pub fn new_from_trusted_setup(trusted_setup: TrustedSetup) -> Result<Self, Err
let peerdas_trusted_setup = PeerDASTrustedSetup::from(&trusted_setup);
let context = DASContext::new(
&peerdas_trusted_setup,
rust_eth_kzg::UsePrecomp::Yes {
width: rust_eth_kzg::constants::RECOMMENDED_PRECOMP_WIDTH,
},
);
```

```
Ok(Self {
trusted_setup: KzgSettings::load_trusted_setup(
&trusted_setup.g1_points(),
&trusted_setup.g2_points(),
)?,
context,
})
}
}
```

핵심 KZG 연산

1. 블롭에서 커밋먼트 생성

```
impl Kzg {
/// 블롭으로부터 KZG 커밋먼트 계산
pub fn blob_to_kzg_commitment(&self, blob: &Blob) -> Result<KzgCommitment, Err
c_kzg::KzgCommitment::blob_to_kzg_commitment(blob, &self.trusted_setup)
.map(|commitment| KzgCommitment(commitment.to_bytes().into_inner()))
.map_err(Into::into)
}
}
```

동작 과정:

1. 블롭 데이터를 다항식으로 해석
2. 신뢰할 수 있는 설정의 G1 포인트들과 내적 계산
3. 결과를 KzgCommitment로 래핑

2. KZG 증명 생성

```
impl Kzg {
/// 블롭과 커밋먼트에 대한 KZG 증명 계산
pub fn compute_blob_kzg_proof(
&self,
blob: &Blob,
kzg_commitment: KzgCommitment,
) -> Result<KzgProof, Error> {
c_kzg::KzgProof::compute_blob_kzg_proof(
blob,
&kzg_commitment.into(),
&self.trusted_setup
)
.map(|proof| KzgProof(proof.to_bytes().into_inner()))
.map_err(Into::into)
}
}
```

증명 생성 과정:

1. 블롭 데이터를 다항식 f(x)로 변환
2. 랜덤 평가점 z에서 f(z) 계산
3. 몫 다항식 q(x) = (f(x) - f(z)) / (x - z) 계산
4. q(x)에 대한 커밋먼트를 증명으로 반환

3. KZG 증명 검증

```
impl Kzg {
/// KZG 증명 검증
pub fn verify_blob_kzg_proof(
&self,
blob: &Blob,
kzg_commitment: KzgCommitment,
kzg_proof: KzgProof,
) -> Result<(), Error> {
if !c_kzg::KzgProof::verify_blob_kzg_proof(
blob,
&kzg_commitment.into(),
&kzg_proof.into(),
&self.trusted_setup,
)? {
Err(Error::KzgVerificationFailed)
} else {
```

```
Ok(())
}
}
}
```

검증 과정:

1. 블롭으로부터 커밋먼트 재계산
2. 제공된 커밋먼트와 비교
3. 페어링 연산을 통한 증명 검증

4. 배치 검증

```
impl Kzg {
/// 여러 블롭-커밋먼트-증명 쌍을 배치로 검증
pub fn verify_blob_kzg_proof_batch(
&self,
blobs: &[Blob],
kzg_commitments: &[KzgCommitment],
kzg_proofs: &[KzgProof],
) -> Result<(), Error> {
let commitments_bytes = kzg_commitments
.iter()
.map(|comm| Bytes48::from(*comm))
.collect::<Vec<_>>();
```

```
let  proofs_bytes = kzg_proofs
.iter()
.map(|proof| Bytes48::from(*proof))
.collect::<Vec<_>>();


if !c_kzg::KzgProof::verify_blob_kzg_proof_batch(
blobs,
&commitments_bytes,
&proofs_bytes,
&self.trusted_setup,
)? {
Err(Error::KzgVerificationFailed)
} else {
Ok(())
}
}
}
```

배치 검증의 장점:

- **성능 향상**: 개별 검증보다 약 2-3배 빠름
- **페어링 최적화**: 여러 페어링을 하나로 결합
- **메모리 효율성**: 중간 결과 재사용

PeerDAS 지원

셀 계산 및 증명

```
impl Kzg {
/// 블롭으로부터 셀과 증명들 계산
pub fn compute_cells_and_proofs(
&self,
blob: KzgBlobRef<'_>,
) -> Result<CellsAndKzgProofs, Error> {
let (cells, proofs) = self
.context()
.compute_cells_and_kzg_proofs(blob)
.map_err(Error::PeerDASKZG)?;

let c_kzg_proof = proofs.map(KzgProof);
Ok((cells, c_kzg_proof))
}
}
```

PeerDAS 개념:

- **셀 (Cell)**: 블롭을 작은 조각으로 분할한 단위
- **확장 블롭**: 리드-솔로몬 코딩으로 확장된 블롭
- **데이터 가용성**: 일부 셀만으로도 전체 블롭 복구 가능

셀 검증

```
impl Kzg {
/// 셀-증명-커밋먼트 쌍들을 배치로 검증
pub fn verify_cell_proof_batch(
&self,
cells: &[CellRef<'_>],
kzg_proofs: &[Bytes48],
columns: Vec<CellIndex>,
kzg_commitments: &[Bytes48],
) -> Result<(), Error> {
let verification_result = self.context().verify_cell_kzg_proof_batch(
commitments.to_vec(),
columns,
cells.to_vec(),
proofs.to_vec(),
);

match verification_result {
Ok(_) => Ok(()),
Err(e) if e.invalid_proof() => Err(Error::KzgVerificationFailed),
Err(e) => Err(Error::PeerDASKZG(e)),
}
```

```
}

}
```

셀 복구

```
impl Kzg {

/// 일부 셀들로부터 전체 셀 집합과 증명들 복구

pub fn recover_cells_and_compute_kzg_proofs(

&self,

cell_ids: &[u64],

cells: &[CellRef<'_>],

) -> Result<CellsAndKzgProofs, Error> {

let (cells, proofs) = self

.context()

.recover_cells_and_kzg_proofs(cell_ids.to_vec(), cells.to_vec())

.map_err(Error::PeerDASKZG)?;


let c_kzg_proof = proofs.map(KzgProof);

Ok((cells, c_kzg_proof))

}

}
```

Lighthouse 통합

BeaconBlockBody에서의 사용

```
// consensus/types/src/beacon_block_body.rs
pub type KzgCommitments<E> =
VariableList<KzgCommitment, <E as EthSpec>::MaxBlobCommitmentsPerBlock>;

#[superstruct(only(Deneb, Electra, Fulu))]
pub blob_kzg_commitments: KzgCommitments<E>,
```

블록 구조에서의 역할:

- **Deneb 이후**: 모든 블록에 KZG 커밋먼트 필드 포함
- **최대 개수**: 네트워크별로 설정된 최대 블롭 수 제한
- **SSZ 직렬화**: 블록 직렬화에 포함

버전화된 해시 계산

```
// crypto/kzg/src/kzg_commitment.rs
pub const VERSIONED_HASH_VERSION_KZG: u8 = 0x01;

impl KzgCommitment {
pub fn calculate_versioned_hash(&self) -> Hash256 {
let mut versioned_hash = hash_fixed(&self.0);
versioned_hash[0] = VERSIONED_HASH_VERSION_KZG;
Hash256::from_slice(versioned_hash.as_slice())
```

```
}

}
```

버전화된 해시 용도:

- **실행 레이어 연동**: 트랜잭션의 블롭 참조
- **가스 계산**: 블롭 가스 사용량 계산
- **검증**: 블롭과 트랜잭션 간 일치성 확인

실행 레이어 통합

```rust
// beacon_node/execution_layer/src/engine_api/new_payload_request.rs
pub fn verify_versioned_hashes(&self) -> Result<(), Error> {
if let Ok(versioned_hashes) = self.versioned_hashes() {
// 블록의 KZG 커밋먼트들로부터 버전화된 해시 계산
let expected_hashes: Vec<_> = block_ref
.body
.blob_kzg_commitments
.iter()
.map(kzg_commitment_to_versioned_hash)
.collect();
// 실행 페이로드의 해시들과 비교
if versioned_hashes != expected_hashes {
return Err(Error::InvalidVersionedHashes);
}
```

```
}

Ok(())

}
```

에러 처리

KZG 에러 타입

```
// crypto/kzg/src/lib.rs

#[derive(Debug)]

pub  enum Error {

/// C-KZG 라이브러리 에러

Kzg(c_kzg::Error),

/// PeerDAS KZG 에러

PeerDASKZG(rust_eth_kzg::Error),

/// KZG 검증 실패

KzgVerificationFailed,

/// 배열 길이 불일치

InconsistentArrayLength(String),

/// 데이터 복구 실패

ReconstructFailed(String),

/// DAS 컨텍스트 미초기화

DASContextUninitialized,
```

```
}
```

에러 처리 전략:

- **라이브러리 에러**: 하위 라이브러리 에러를 래핑
- **검증 실패**: 명확한 실패 원인 제공
- **복구 가능**: 가능한 경우 복구 방법 제시

성능 최적화

벤치마크 결과

```
// crypto/kzg/benches/benchmark.rs
pub fn bench_init_context(c: &mut Criterion) {
c.bench_function("Initialize context rust_eth_kzg", |b| {
b.iter(|| {
DASContext::new(
&trusted_setup,
rust_eth_kzg::UsePrecomp::Yes {
width: rust_eth_kzg::constants::RECOMMENDED_PRECOMP_WIDTH,
},
)
})
});
}
```

성능 특성:

- **초기화 시간**: 신뢰할 수 있는 설정 로딩 약 100ms
- **커밋먼트 계산**: 블롭당 약 10ms
- **증명 생성**: 블롭당 약 50ms
- **검증**: 개별 약 5ms, 배치 약 2ms/개

메모리 사용량

```
// 신뢰할 수 있는 설정 크기
const G1_POINTS: usize = 4096; // 48 bytes each
const G2_POINTS: usize = 65; // 96 bytes each
const TOTAL_SIZE: usize = G1_POINTS * 48 + G2_POINTS * 96; // ~202KB
```

메모리 최적화:

- **사전 계산**: 자주 사용되는 값들 미리 계산
- **캐싱**: 계산 결과 캐싱으로 중복 계산 방지
- **배치 처리**: 메모리 지역성 활용

보안 고려사항

신뢰할 수 있는 설정

```
// 신뢰할 수 있는 설정 검증
impl TrustedSetup {
pub fn validate(&self) -> Result<(), Error> {
```

```
// G1 포인트들이 유효한 곡선 위의 점인지 확인
for point in &self.g1_points {
if !is_valid_g1_point(&point.0) {
return Err(Error::InvalidG1Point);
}
}
// G2 포인트들 검증
for point in &self.g2_points {
if !is_valid_g2_point(&point.0) {
return Err(Error::InvalidG2Point);
}
}
Ok(())
}
}
```

보안 요구사항:

- **신뢰할 수 있는 설정**: 신뢰할 수 있는 의식(ceremony)으로 생성
- **포인트 검증**: 모든 포인트가 유효한 곡선 위의 점인지 확인
- **무결성**: 설정 파일의 해시 검증

사이드 채널 공격 방지

```
// 상수 시간 연산 보장
```

```rust
impl KzgCommitment {
pub fn constant_time_eq(&self, other: &Self) -> bool {
use subtle::ConstantTimeEq;
self.0.ct_eq(&other.0).into()
}
}
```

보안 조치:

- **상수 시간**: 타이밍 공격 방지를 위한 상수 시간 연산
- **메모리 보호**: 민감한 데이터의 안전한 메모리 관리
- **입력 검증**: 모든 입력에 대한 철저한 검증

테스트 및 검증

단위 테스트

```rust
#[cfg(test)]
mod tests {
use super::*;
#[test]
fn test_kzg_commitment_versioned_hash() {
let commitment = KzgCommitment::from_str(
"0x53fa09af35d1d1a9e76f65e16112a9064ce30d1e4e2df98583f0f5dc2e7dd13a4f421a9c89f51
).unwrap();
```

```
let versioned_hash = commitment.calculate_versioned_hash();
assert_eq!(versioned_hash.as_bytes()[0], VERSIONED_HASH_VERSION_KZG);
}
#[test]
fn test_blob_to_commitment_roundtrip() {
let kzg = setup_kzg();
let blob = generate_test_blob();
let commitment = kzg.blob_to_kzg_commitment(&blob).unwrap();
let proof = kzg.compute_blob_kzg_proof(&blob, commitment).unwrap();
assert!(kzg.verify_blob_kzg_proof(&blob, commitment, proof).is_ok());
}
}
```

통합 테스트

```
#[test]
fn test_peerdas_cell_operations() {
let kzg = setup_kzg_with_das();
let blob = generate_test_blob();
// 셀과 증명 생성
let (cells, proofs) = kzg.compute_cells_and_proofs(&blob).unwrap();
// 일부 셀만 선택
let selected_cells: Vec<_> = cells[..cells.len()/2].iter().collect();
```

```
let selected_ids: Vec<_> = (0..cells.len()/2).map(|i| i as u64).collect();

// 셀 복구

let (recovered_cells, recovered_proofs) = kzg

.recover_cells_and_compute_kzg_proofs(&selected_ids, &selected_cells)

.unwrap();

assert_eq!(recovered_cells, cells);

}
```

결론

Lighthouse의 KZG 커밋먼트 구현은 Ethereum의 데이터 가용성 솔루션의 핵심 구성 요소입니다. 이 구현은 다음과 같은 특징을 가집니다:

주요 강점

1. **이중 라이브러리 지원**: C-KZG와 Rust-ETH-KZG를 모두 활용하여 최적의 성능과 기능성 제공
2. **PeerDAS 준비**: 미래의 데이터 가용성 샘플링 프로토콜 지원
3. **타입 안전성**: Rust의 타입 시스템을 활용한 안전한 API 설계
4. **성능 최적화**: 배치 검증과 사전 계산을 통한 성능 향상
5. **표준 준수**: Ethereum 표준과 완전한 호환성

기술적 혁신

- **효율적인 블롭 처리**: 대용량 데이터의 효율적인 커밋먼트와 검증
- **확장 가능한 아키텍처**: PeerDAS와 같은 미래 프로토콜 지원
- **강력한 암호학적 보장**: KZG 다항식 커밋먼트의 수학적 보장 활용

이러한 구현을 통해 Lighthouse는 Ethereum의 확장성 로드맵에서 중요한 역할을 하는 데이터 가용성 솔루션을 안정적이고 효율적으로 지원할 수 있습니다.